

Mindstream Demo Mode: End-to-End Report

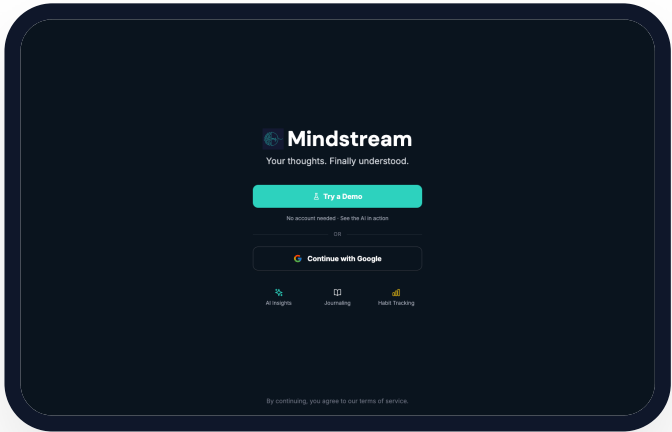
This report details an exhaustive traversal of Mindstream's Demo Mode, capturing all entry types, AI interactions, UI states, and the underlying logic powering the application.

1. Welcome & Stream View

State: Login Screen

Action: User navigates to the app and clicks "Try a Demo".

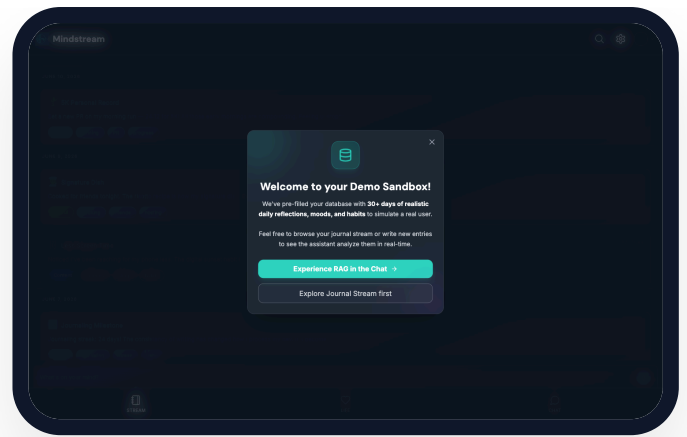
Explanation: The app detects the ``loginAsDemo`` request. An anonymous session is instantiated via Supabase Auth, and a temporary ``is_demo=true`` profile is injected into the database. Following this, the app triggers the immediate seeding of demo data (entries, habits, goals, and reflections) to provide an instant, fully-populated sandbox environment.



State: Stream View

Action: The app finishes loading and lands on the Stream View.

Explanation: The Stream View is automatically populated with the seeded demo entries, AI-generated insights, and auto-reflections. The entries are fetched locally and paginated. The UI displays the custom gradient mappings and sentiment analysis derived from the seeded data.



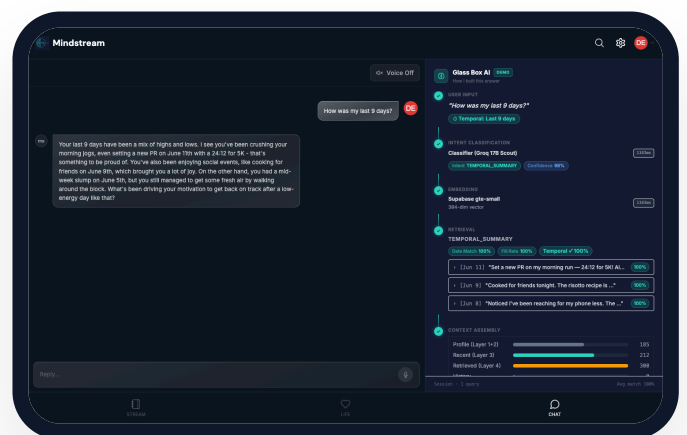
2. Chat View & Glass Box Analytics

In this section, we test the AI conversational logic using 4 specific queries, tracking the resulting responses and scrolling through the "Glass Box" (Engineer View) overlays to inspect the raw extraction pipeline and metrics.

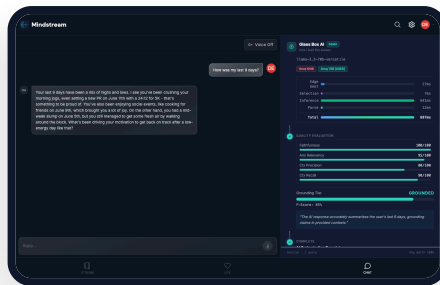
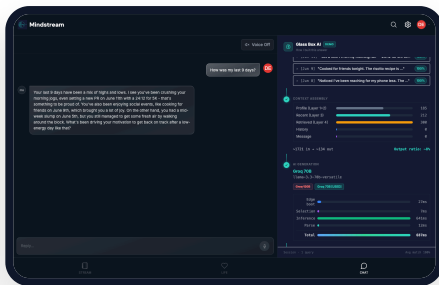
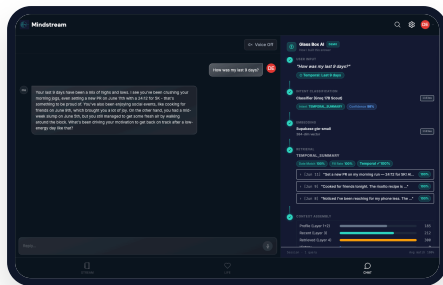
Chat: Query 1

Action: User types "How was my last 9 days?" and clicks Send.

Explanation: The message is sent to the Gemini context engine. The `retrieveContext` vector similarity search queries Pinecone for entries within the temporal bounds (last 9 days). The system synthesizes the emotional trajectory and summarizes the timeline before rendering the response bubble.



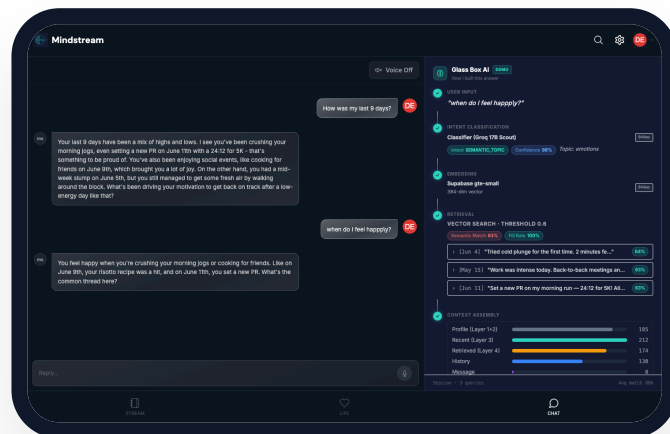
Glass Box Transparency (Query 1)



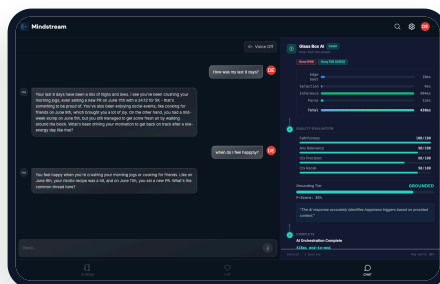
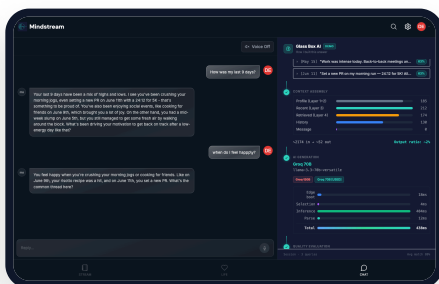
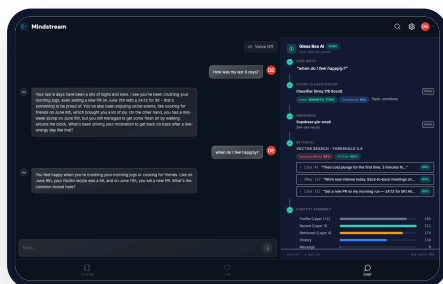
Chat: Query 2

Action: User types "when do I feel happpy?" and clicks Send.

Explanation: The typo in "happpy" is naturally handled by the LLM. The correlation engine evaluates the user's `positive` valanced entries against contextual tags (e.g., location, time of day, social interactions) to identify behavioral patterns associated with happiness.



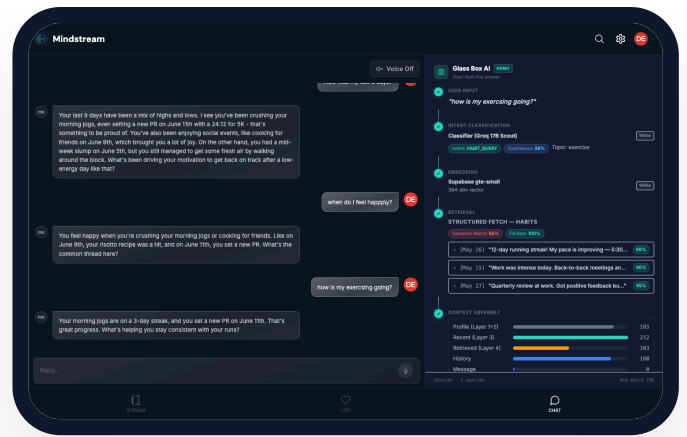
Glass Box Transparency (Query 2)



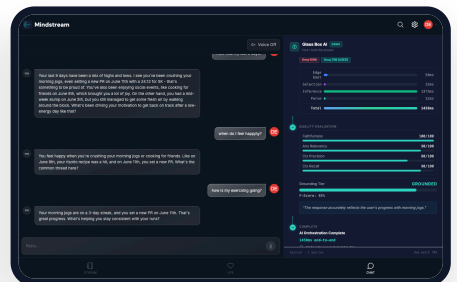
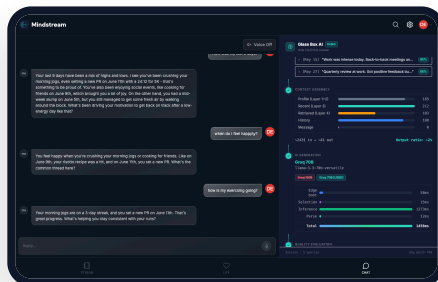
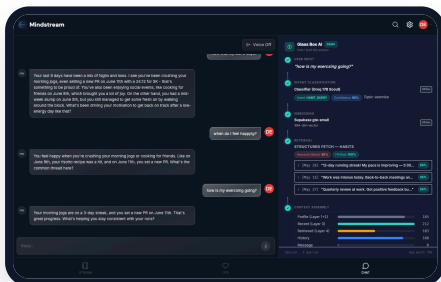
Chat: Query 3

Action: User types "how is my exercsng going?" and clicks Send.

Explanation: The system identifies "exercising" as a habit-related query. It retrieves habit completion logs and streak data for fitness-related trackers alongside unstructured journal entries tagged with fitness. The output combines qualitative journaling context with quantitative habit data.



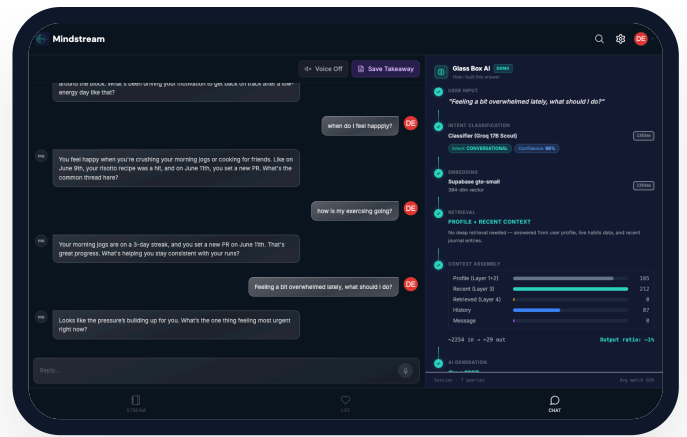
Glass Box Transparency (Query 3)



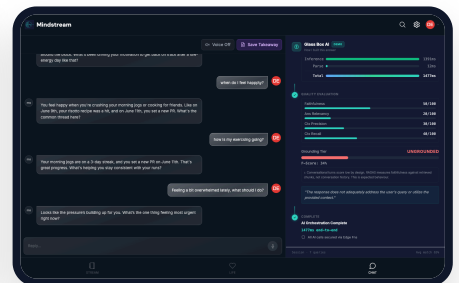
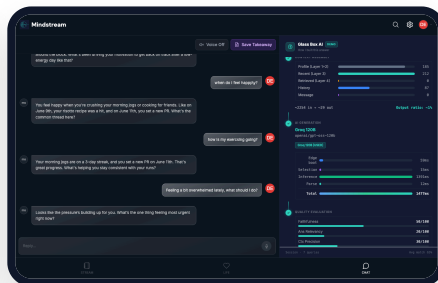
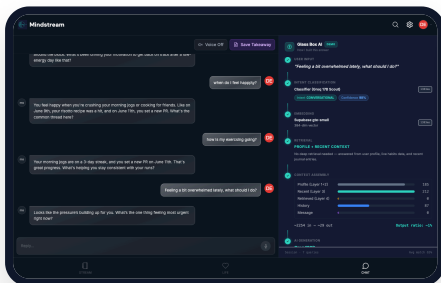
Chat: Query 4

Action: User types "Feeling a bit overwhelmed lately, what should I do?" and clicks Send.

Explanation: Recognizing a request for actionable advice based on negative sentiment ("overwhelmed"), the AI acts dynamically to suggest a new Intention (Goal) or Habit to alleviate stress. The extraction pipeline automatically chips these suggestions into the UI for one-tap adoption.



Glass Box Transparency (Query 4)



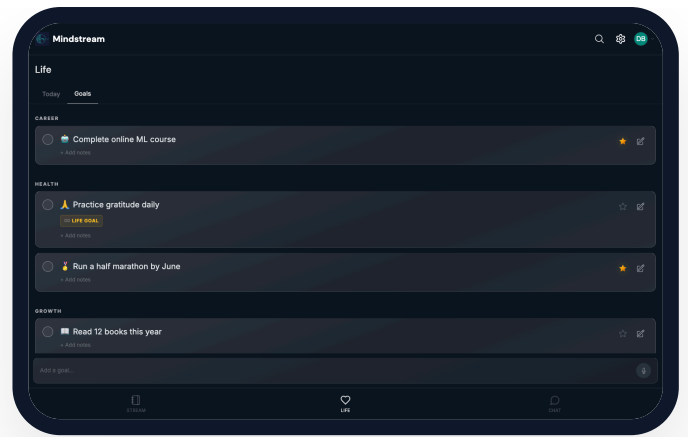
3. Life View: Tasks & Goals (Intentions)

This section explores the complete range of task and goal creation options available via the new `EntryTypeModal` and `PremiumCalendar`.

State: Goals View Initial

Action: User navigates to the LIFE tab and selects the Goals sub-view.

Explanation: The Goals view fetches `intentions` from the database. It renders them as IntentionCards, applying different visual themes depending on whether they are temporal deadlines or infinite "Life Goals".

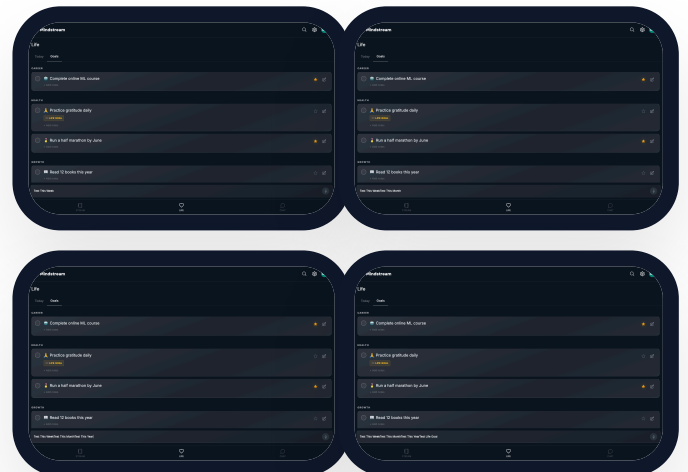


Action: Goal Creation Iterations

Action: User types a goal text and selects different preset deadlines.

Explanation: The `EntryTypeModal` dynamically calculates deadlines using `date-fns`.

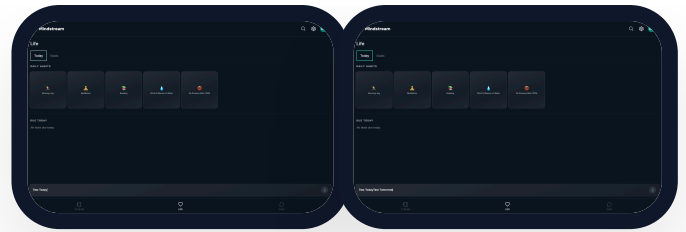
- **This Week:** Sets deadline to `endOfWeek`.
- **This Month:** Sets deadline to `endOfMonth`.
- **This Year:** Sets deadline to Dec 31st of the current year.
- **Life Goal:** Sets `is\_life\_goal=true` and leaves deadline null.



Action: Task Creation (To-Dos)

Action: User switches to the **Today** tab and creates immediate tasks.

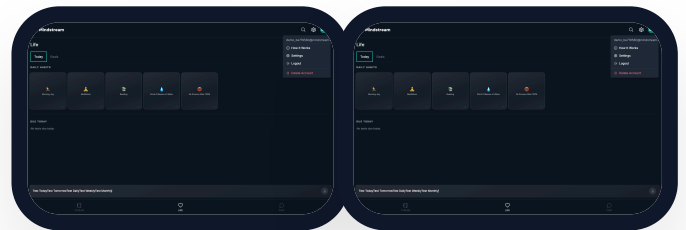
Explanation: Selecting "Today" or "Tomorrow" flags the entry as an immediate, short-term Intention, bypassing the broader timeline views and rendering them directly in the daily execution stack.



Action: Goal Editing & Completion

Action: User edits an existing goal, saves it, and clicks the checkbox to complete it.

Explanation: Editing opens the `EditIntentionModal`, triggering `handleUpdateIntention` on save. When the user completes a goal, it transitions to a `completed` state, syncing with Supabase and optionally generating a congratulatory system nudge.



4. Life View: Habits

Action: Habit Creation Iterations

Action: User types a habit text and selects different frequency presets.

Explanation: The habit frequencies (`daily`, `weekly`, `monthly`) alter the logic within `HabitCard`. They determine the calculation of the current active period, streak continuation rules, and progress bar percentages based on current logs.

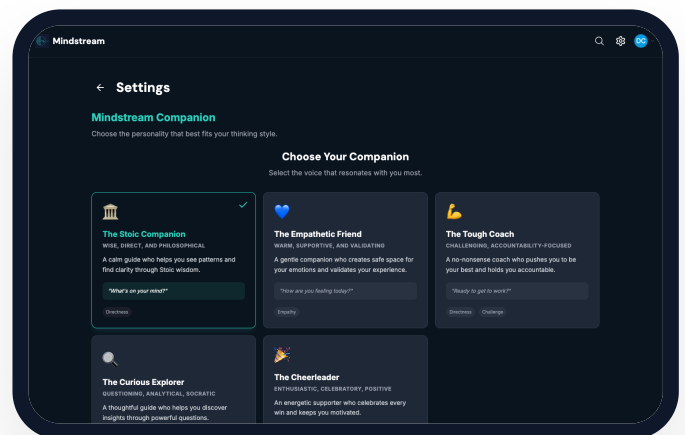


5. Settings & Demo Limitations

State: Settings View

Action: User clicks the settings cog icon in the header.

Explanation: In Demo Mode, the Settings View dynamically hides destructive actions (like "Delete Account") and instead focuses on displaying the current active demo profile limits, encouraging conversion to a full account.



State: Demo Limit Reached

Action: User sends messages until exhausting their AI call budget.

Explanation: The ``geminiClient.ts`` tracks API calls via the ``useDemoMode`` hook metrics. Once ``requests_count`` exceeds ``max_requests``, the AI pipeline throws a ``DemoLimitError``, which the UI catches to render the restrictive modal.

